

Process Engineering

ENGR-3027

Lab #2

Logic Thinking for Robotic Control

Importance of Programming Logic

- Robotic programming logic is important in engineering and manufacturing because it's what makes a robot behave correctly, safely, and consistently in the real world.
- Effective programming logic tells the robot when to move, what to check, and what to do if something changes.
- Robotic programming logic is the “brain” of automation. It connects engineering design, sensors, and machines into a system that can run safely, accurately, and continuously on a manufacturing floor.

Importance of Programming Logic

Why it matters

- **Reliable, repeatable production**

- Robots must do the same job thousands of times with the same results (pick-and-place, welding, painting, assembly).
- Logic controls the exact sequence, so every cycle happens the same way.

- **Decision-making using sensors**

- Robots use sensors (vision, proximity, force/torque, encoders) and need logic to respond.
- Example: *If* a part is missing or misaligned, *then* stop, retry, or reject the part.

Importance of Programming Logic

- **Safety for people and equipment**
 - Robots can move fast and with high force. Logic is used for safety interlocks, emergency stop behavior, and safe zones.
 - Example: If a safety gate opens, then motion stops immediately.
- **Higher quality and less scrap**
 - Logic allows checks during the process (dimensions, position, torque, weld time) so problems are caught early.
 - Example: If a measurement is out of tolerance, then reject and log it before it reaches the customer.
- **Efficiency and speed**
 - Good logic reduces wasted motion and downtime.
 - Robots can loop through tasks quickly, coordinate with conveyors, and keep the line moving.

Importance of Programming Logic

- **Flexibility and scaling**

- Well-designed logic makes it easier to change products or add new steps.
- Example: switching between “Product A” and “Product B” using a mode/recipe selection.

- **Troubleshooting and maintenance**

- Clear logic helps technicians find faults faster.
- Example: the program can show exactly where the robot stopped and what condition wasn't met.

Loops

- In robotics, loops are programming tools that let a robot repeat the same steps over and over without rewriting the code.
- This is useful for tasks the robot does repeatedly like moving forward, checking sensors, or controlling motors.
- A loop can run a set number of times or keep running until a condition is met, which is why loops are so important for continuous robot actions and quick responses.

Loops

Key Functions & Examples

- **Automating repetition:** Instead of writing “move forward” five times, you can use a loop to repeat that action automatically with much less code.
- **Continuous operation (forever loops):** A **forever loop** keeps running nonstop. This is useful for robots that must always stay alert like a robotic sweeper that constantly checks for obstacles and turns when needed.
- **Repeating until something happens:** Some loops run **until a condition changes**.
Example: A robot drives forward **until** a bumper switch is pressed.
- **Handling user input:** In remote-controlled robots, loops repeatedly check the joystick or buttons so the robot can respond quickly.
- **Feedback control:** In more advanced robots, loops are used in layers.
Example: One loop handles the big goal (like “follow the path”), while another loop handles detailed motor control to keep movement smooth and accurate.

Loops

Types of Loops

- **For loop:** Repeats something a **set number of times**.
Example: “Take 5 steps.”
- **While loop:** Keeps repeating **as long as a condition is true**.
Example: “Move forward while the path is clear.”
- **Forever loop:** Repeats **non-stop** until the program is stopped.
Example: “Keep checking sensors and driving.”

Loops

Why Loops Are Important

- **Efficiency:** Loops save time and code because you don't have to write the same command over and over.
- **Organization:** They make your program easier to read and keep everything cleaner.
- **Scalability:** It's simple to change the loop to work for a different task by simply adjusting a number or condition.
- **Real-world use:** Loops help robots behave consistently and react to what's happening, whether it's a simple repeated task or something complex like navigation.

Conditionals

- In robot programming, conditionals are instructions that help a robot make decisions.
- They check whether something is true or false, and then the robot chooses what to do next.
- This lets the robot react to its environment or sensor readings, instead of always doing the exact same steps in the same order.

Conditionals

How Conditionals Work in Robotics

In robotics, conditionals use “**if-then**” logic to help a robot decide what to do.

- **If** a condition is true, **then** the robot runs a specific set of instructions (a block of code).
- **If** the condition is false, the robot **skips** that block and either continues with the rest of the program or runs a **different** set of instructions (an alternative action).

The condition being checked is usually a **Boolean expression**, which means it can only have two results:

- **True** (often shown as **1**)
- **False** (often shown as **0**)

Conditionals

Common Types of Conditional Statements

Most programming languages (including robot programming) give you a few common ways to make decisions:

- **If statements:** The simplest type. The robot runs a block of code **only if** the condition is **true**.
Example: “If the button is pressed, start the motor.”
- **If-Else statements:** Gives the robot **two choices**. If the condition is true, it does one action; **otherwise**, it does a different action.
Example: “If an obstacle is detected, turn; **else** keep moving forward.”
- **If-Else If-Else (chained conditionals):** Used when you have **more than two options**. The robot checks conditions **in order** until it finds one that’s true, runs that code, and then moves on.
Example: “If the line is left, turn left; else if the line is right, turn right; else go straight.”
- **Switch statements:** A cleaner way to handle **many possible values** for one variable, instead of writing a long chain of if-else if statements.
Example: “Based on the robot’s mode (1, 2, 3...), choose the correct behavior.”

Conditionals

Conditionals help robots **act on their own** by making choices based on what their sensors detect.

Here are some common examples:

- **Obstacle Avoidance:** A robot can use an IR (infrared) sensor to check for something in its way.
Example: If the **left** sensor detects an obstacle, the robot turns **right**. If the **right** sensor detects an obstacle, it turns **left**.
- **Navigation:** A drone can use GPS to decide what to do.
Example: If it's **at the destination**, it **lands**. If not, it **keeps flying**.
- **Task Automation:** A farming robot can use a camera/vision system to identify plants.
Example: If the plant is a **weed**, remove it. If it's a **crop**, leave it and keep checking.
- **Safety Protocols:** In factories, robots use safety sensors to protect people.
Example: If a **safety gate is open**, the robot **stops**. If the gate is closed, it can **run normally**.

Overall, conditionals give robots the ability to **respond to real-world situations** instead of blindly following one set of steps.

Teach Pendants

- Teach pendants are important because they're the main hands-on interface for setting up, programming, and safely controlling an industrial robot.
- Think of the pendant as the robot's "remote control and programming keypad" used on the shop floor.
- A teach pendant is essential because it lets humans safely move the robot, teach it real positions, build/edit programs, and diagnose problems which is exactly what you need for reliable robot operation in engineering and manufacturing.

Teach Pendants

Why teach pendants matter

- **Teaching positions (points)**

- Most robot programs are built from real robot positions: pick point, place point, approach point, home, etc.
- The pendant lets you **jog** the robot into the exact spot, then **record** that position into the program.

- **Safe manual control during setup**

- When you're close to a robot, safety is critical. Pendants usually include:
 - an **E-stop** button
 - a **deadman/enable switch** (robot moves only while you hold it correctly)
 - **reduced-speed “teach” mode**
- This allows careful testing without the robot moving at full production speed.

Teach Pendants

- **Programming and editing**

- You can create and modify programs directly: moves, speeds, waits, loops, if-statements, I/O checks, etc.
- Quick for small changes on the line (ex: tweak a pickup height by 2 mm).

- **Calibration and coordinate setup**

- Pendants are used to set things like:
 - Tool Center Point (TCP) (the “tip” of the gripper or torch)
 - work coordinates / frames (where the part or fixture is located)
- Good calibration provides accurate, repeatable motion.

Teach Pendants

- **Testing and troubleshooting**

- You can step through code, run one line at a time, and see robot status and alarms.
- You can also test inputs/outputs (sensors, grippers, valves) to find faults faster.

- **Production control**

- Operators often use the pendant to start/stop jobs, select programs or recipes, and monitor the robot.